SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

Code of Conduct:

I **B Sandhya(192311292)** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sandhya

RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

ANSWERS

1.Reverse Engineering Task: ATM Withdrawal Module

1. Introduction

Reverse engineering in software testing involves analyzing a system's observable inputs and outputs to understand its functional behavior and reconstruct its possible testing strategy. In this task, an ATM withdrawal module is partially tested, and only its external behavior is known, including withdrawal limits, balance validation, and error messages. The objective is to identify the black-box testing techniques used, evaluate their adequacy, and design an improved set of test cases.

2. Analysis of Observed System Behavior

The ATM withdrawal module accepts a withdrawal amount and checks whether the transaction satisfies the system's rules.

The expected behavior can be inferred as follows:

- If the withdrawal amount is valid and sufficient funds are available, the transaction succeeds.
- If the amount exceeds the permitted withdrawal limit, the system rejects the transaction.
- If the withdrawal amount exceeds the available balance, an insufficient-balance error is displayed.
- If the amount is zero, negative, or incorrectly formatted, the system rejects the input.
- If the withdrawal amount exceeds the account's daily withdrawal limit, the transaction is rejected.
- If the transaction is successful, the account balance is reduced by the withdrawal amount.

These observations help reconstruct the expected functionality without examining the source code.

3. Functional Decomposition

The ATM withdrawal module can be divided into the following functional components.

Component	Function	Possible Failure
Input Validation	Checks the entered withdrawal amount	Accepts zero or negative values
Withdrawal Limit Validation	Checks the maximum permitted withdrawal	Allows excessive withdrawals
Balance Validation	Checks whether sufficient funds are available	Allows withdrawal beyond the balance
Daily Limit Validation	Checks total withdrawals for the day	Allows the daily limit to be exceeded
Transaction Processing	Deducts the approved amount	Deducts an incorrect amount
Error Handling	Displays appropriate error messages	Displays incorrect or unclear messages
Balance Update	Updates the account balance after success	Updates the balance incorrectly
Transaction Logging	Records the transaction result	Creates missing or duplicate records

4. Equivalence Class Partitioning (ECP)

Equivalence Class Partitioning divides inputs into valid and invalid groups expected to produce similar behavior.

Assumptions: The minimum withdrawal is ₹100, the maximum per transaction is ₹25,000, and the account balance and daily limit must also be sufficient. These values are illustrative; actual ATM rules may differ.

Class	Input Condition	Example	Expected Result
EC1	Valid withdrawal amount	₹500	Transaction succeeds if other conditions are satisfied
EC2	Amount below minimum	₹50	Reject withdrawal
EC3	Zero withdrawal	₹0	Reject withdrawal
EC4	Negative withdrawal	-₹500	Reject withdrawal
EC5	Amount above transaction limit	₹30,000	Reject withdrawal
EC6	Amount exceeds available balance	₹5,000 with ₹3,000 balance	Display insufficient-balance error
EC7	Amount exceeds remaining daily limit	₹5,000 when only ₹2,000 remains	Reject withdrawal
EC8	Invalid input format	Alphabetic characters	Display input validation error
EC9	Missing input	Blank amount	Request a valid amount

5. Boundary Value Analysis (BVA)

Boundary Value Analysis tests values at and around the permitted limits to identify boundary-related defects.

Assuming the permitted withdrawal range is ₹100–₹25,000:

Test Case	Withdrawal Amount	Boundary Condition	Expected Result
BVA1	₹99	Just below minimum	Reject
BVA2	₹100	Minimum boundary	Accept if other conditions are satisfied
BVA3	₹101	Just above minimum	Accept if other conditions are satisfied
BVA4	₹24,999	Just below maximum	Accept if other conditions are satisfied
BVA5	₹25,000	Maximum boundary	Accept if other conditions are satisfied
BVA6	₹25,001	Just above maximum	Reject

Additional tests should check the available balance and remaining daily limit at their boundaries.

6. Inferred Black-Box Testing Techniques

Based on the observed behavior, the following techniques may have been used:

1. Equivalence Class Partitioning: Inputs may have been grouped into valid and invalid withdrawal amounts.

2. Boundary Value Analysis: Values near minimum and maximum withdrawal limits may have been tested.

3. Decision Table Testing: Combinations of withdrawal amount, available balance, and daily limit may have been evaluated.

4. Error Guessing: Testers may have checked negative amounts, zero values, insufficient balances, and invalid input formats.

However, the available observations do not prove which techniques were actually used. They only indicate which techniques could explain the existing test coverage.

7. Evaluation of Testing Adequacy

Testing only normal withdrawal amounts is insufficient because several important scenarios may remain untested.

- Boundary values may not have been checked systematically.
- Negative and zero withdrawal amounts may bypass validation.
- Interactions between balance limits and withdrawal limits may be missed.
- Daily withdrawal limits may not be validated correctly.
- Error messages may be inaccurate or inconsistent.
- Failed transactions may incorrectly modify the account balance.
- Concurrent or repeated requests may result in duplicate withdrawals.

Therefore, a combination of equivalence partitioning, boundary value analysis, decision tables, and negative testing is required.

8. Improved Test Case Design

The following test cases provide broader functional coverage.

Test ID	Scenario	Input / Condition	Expected Result
TC01	Valid withdrawal	₹500; balance ₹5,000	Success; balance becomes ₹4,500
TC02	Minimum valid amount	₹100; sufficient balance and daily limit	Success
TC03	Below minimum	₹99	Rejected
TC04	Zero amount	₹0	Rejected
TC05	Negative amount	-₹500	Rejected
TC06	Maximum valid amount	₹25,000; sufficient balance and daily limit	Success
TC07	Above transaction limit	₹25,001	Rejected
TC08	Insufficient balance	₹2,000 withdrawal; balance ₹1,000	Rejected; balance unchanged
TC09	Exact balance withdrawal	₹1,000 withdrawal; balance ₹1,000	Success if zero balance afterward is permitted
TC10	Daily limit exceeded	Withdrawal exceeds remaining daily allowance	Rejected

Test ID	Scenario	Input / Condition	Expected Result
TC11	Invalid input format	Alphabetic input	Validation error
TC12	Blank input	No amount entered	Validation error
TC13	Failed transaction	Any rejected withdrawal	Balance remains unchanged
TC14	Repeated request	Same withdrawal request submitted twice	No unintended duplicate debit
TC15	Concurrent withdrawals	Two requests compete for the same balance	No overdraft or inconsistent balance
TC16	System interruption	Failure during transaction processing	Transaction is safely completed or rolled back
TC17	Error-message validation	Insufficient balance	Correct and understandable message displayed
TC18	Transaction logging	Successful and failed withdrawals	Accurate transaction records maintained

9. Justification for the Improved Strategy

The improved test design combines multiple black-box techniques to achieve more complete functional coverage.

- **ECP** reduces redundant tests by selecting representative inputs from valid and invalid classes.
- **BVA** detects defects near withdrawal limits and balance boundaries.
- **Decision Table Testing** validates combinations of amount validity, available balance, and daily withdrawal allowance.
- **Negative Testing** verifies that invalid inputs and failed transactions are handled safely.
- **Transaction Testing** checks balance consistency, duplicate requests, and recovery from failures.

Together, these techniques provide stronger evidence that the ATM withdrawal module behaves correctly across normal, boundary, invalid, and exceptional scenarios.

10. Conclusion

Reverse engineering the ATM withdrawal module from its observable behavior helps reconstruct its functional requirements and likely testing strategy. Equivalence Class Partitioning and Boundary Value Analysis provide a foundation for systematic test design, while decision tables and negative testing address interactions and failure conditions. The improved test suite covers critical withdrawal scenarios, protects account balance integrity, and increases confidence in the reliability and security of the ATM system.

2.Reverse Engineering Task: White-Box Testing Approach

1. Introduction

White-box testing examines the internal structure, logic, decisions, loops, and execution paths of a software module. When test documentation is unavailable, reverse engineering can reconstruct the existing testing approach by examining source code and execution traces.

The objectives are to construct a Control Flow Graph (CFG), identify decision points and loops, determine independent execution paths, evaluate existing coverage, and design an improved path-testing strategy.

2. Analysis of Program Logic

Consider the following illustrative code:

```
if (amount > 0) {
   if (balance >= amount) {
      balance = balance - amount;
      System.out.println("Success");
   } else {
      System.out.println("Insufficient balance");
   }
} else {
   System.out.println("Invalid amount");
}

for (int i = 0; i < 3; i++) {
   System.out.println(i);
}
```

The program validates a transaction amount, checks the available balance, processes a successful transaction, and executes a loop three times.

By analyzing the code, we can identify three decision points:

1. amount > 0
2. balance >= amount
3. i < 3

The first two decisions control transaction processing, while the third controls loop execution.

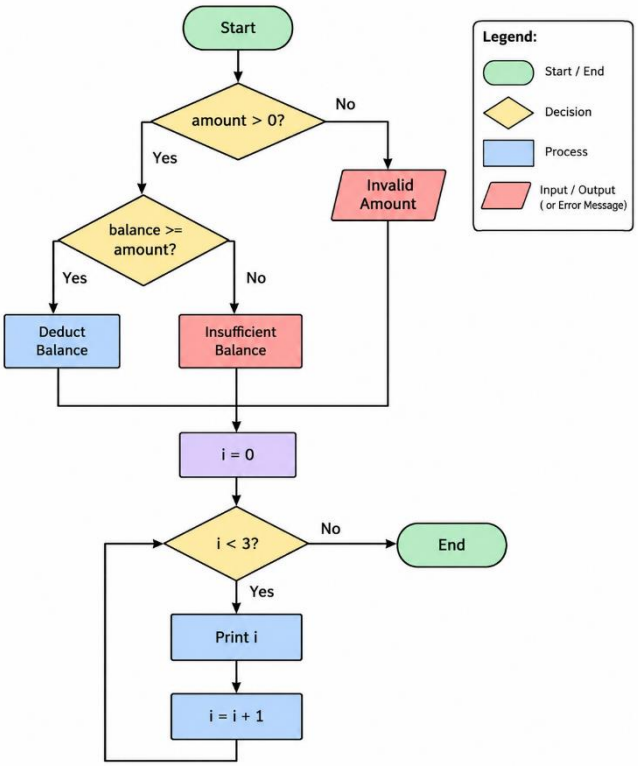
3. Control Flow Graph (CFG)

A Control Flow Graph represents the possible execution flow through a program.

Nodes and execution flow:

- **N1:** Start
- **N2:** Check whether amount > 0
- **N3:** Check whether balance >= amount
- **N4:** Deduct the amount and display success
- **N5:** Display insufficient-balance message
- **N6:** Display invalid-amount message
- **N7:** Initialize loop variable i = 0
- **N8:** Check loop condition i < 3
- **N9:** Display i and increment it
- **N10:** End

Control flow representation:



The graph identifies decision branches, transaction outcomes, and the loop's back edge.

4. Identification of Independent Execution Paths

An independent path introduces at least one new edge that has not appeared in previously selected paths.

The main logical paths are:

- **Path 1:** Valid amount → Sufficient balance → Successful transaction → Loop executes.
- **Path 2:** Valid amount → Insufficient balance → Error message → Loop executes.
- **Path 3:** Invalid amount → Error message → Loop executes.

The loop introduces additional behavior:

- **Loop Path 1:** The loop executes zero times when the initial condition is false.
- **Loop Path 2:** The loop executes once.
- **Loop Path 3:** The loop executes multiple times.
- **Loop Path 4:** The loop terminates after its condition becomes false.

For the given code, the actual loop executes three times because its bound is fixed. The zero-iteration and one-iteration cases require modified test inputs or a configurable loop bound.

5. Cyclomatic Complexity Calculation

Cyclomatic Complexity estimates the number of linearly independent paths in a control flow graph.

The formula is:

V(G) = E − N + 2P

Where:

- E = Number of edges.
- N = Number of nodes.
- P = Number of connected components, normally 1.

For a connected graph, it can also be calculated as:

V(G) = Number of decision points + 1

In the illustrative program, there are three decision points: two if statements and one loop condition.

Therefore:

V(G) = 3 + 1 = 4

This means that four linearly independent paths form a basis for the complete control flow graph.

However, basis-path testing alone does not guarantee that every possible loop iteration count or every combination of input values has been tested.

6. Evaluation of Existing Test Adequacy

Without test documentation, existing coverage must be inferred from execution traces and observed behavior.

The following coverage criteria should be evaluated:

Statement Coverage: Checks whether every executable statement has been executed. It may miss untested decision outcomes.

Branch Coverage: Checks whether each decision outcome has been exercised. It is stronger than statement coverage for identifying missing branches.

Condition Coverage: Checks whether each individual Boolean condition evaluates to both true and false.

Path Coverage: Examines execution paths through the control flow graph. Complete path coverage may be impractical when loops or complex conditions create many possible paths.

Loop Coverage: Checks zero, one, typical, and boundary iteration counts where applicable.

If existing traces show only successful transactions, the insufficient-balance and invalid-amount paths remain inadequately tested. If all statements execute but some branches do not, statement coverage alone has not established adequate decision testing.

7. Improved Path Testing Strategy

The following test cases are proposed for the illustrative program.

Test Case	Amount	Balance	Purpose	Expected Result
TC1	100	500	Valid amount and sufficient balance	Success; balance becomes 400
TC2	600	500	Valid amount but insufficient balance	Error message; balance unchanged
TC3	0	500	Zero amount	Invalid-amount message
TC4	-100	500	Negative amount	Invalid-amount message
TC5	500	500	Amount exactly equals balance	Success; balance becomes zero
TC6	100	1000	Verify normal loop behavior	Loop prints 0, 1, and 2
TC7	100	1000	Repeat execution to verify consistent results	Same output and final state

Additional loop tests should be designed if the loop bound can vary. Test zero iterations, one iteration, typical iterations, and the maximum permitted number of iterations.

The test suite should also verify that failed transactions never change the balance and that repeated execution does not produce unintended side effects.

8. Justification for the Improved Strategy

The improved approach combines several white-box testing techniques:

1. **CFG analysis** reveals the program's control flow and decision branches.
2. **Basis Path Testing** provides a systematic set of independent paths.
3. **Branch Coverage** confirms that both outcomes of each decision are exercised.
4. **Condition Coverage** checks individual Boolean conditions.
5. **Loop Testing** verifies loop initialization, execution, termination, and boundary behavior.
6. **Execution Trace Analysis** compares actual behavior against expected results.

These techniques help reveal missing paths, incorrect conditions, loop defects, and unexpected state changes. They also provide measurable evidence of testing adequacy.

9. Conclusion

Reverse engineering a white-box testing strategy requires examining source code, constructing a Control Flow Graph, identifying decisions and loops, and calculating Cyclomatic Complexity. Basis Path Testing then provides a systematic starting point for selecting independent execution paths.

Combining basis-path, branch, condition, and loop testing produces stronger structural coverage than statement coverage alone. However, complete confidence also requires verifying outputs, boundary values, error handling, and important interactions between program conditions.

3.Reverse Engineering the Testing Approach for a Web Application Login and Transaction System

1. Introduction

A web application login and transaction system allows users to authenticate themselves, view account details, and perform financial transactions. Since the original test documentation is unavailable, the testing approach can be reconstructed by examining defect logs, test results, and application behavior. Black-box techniques help identify functional defects, while white-box techniques help analyze internal logic, branches, and execution paths. A combined testing strategy improves functionality, reliability, security, and transaction accuracy.

2. Analysis of Defect Patterns

Defect logs and test results can provide clues about the testing techniques previously used.

Observed Defect Pattern	Possible Testing Technique	Reasoning
Login fails for certain usernames or passwords	Equivalence Partitioning	Different categories of valid and invalid credentials may have been tested.
Login fails for empty fields or maximum-length inputs	Boundary Value Analysis	Tests may have focused on input limits.
Incorrect error messages for invalid credentials	Negative Testing	Invalid inputs may have been supplied to check error handling.
Transactions fail when the balance equals the transaction amount	Boundary Value Analysis	Exact balance and nearby values need testing.
Incorrect balance updates after transactions	Decision Table Testing or functional testing	Transaction conditions and expected outcomes may have been checked.
Some branches execute incorrectly	Branch Coverage	Different outcomes of conditional statements may have been tested.
Errors occur in specific sequences of login and transaction operations	Path Testing	Execution paths and operation sequences may have been examined.
Duplicate transactions occur after repeated requests	Error Guessing and robustness testing	Repeated requests may expose duplicate-processing defects.

Inference: The defect patterns suggest that functional testing, negative testing, boundary testing, and some structural testing may have been used. However, these techniques cannot be confirmed without the original test cases, code coverage reports, or execution traces.

3. Decomposition into Testable Components

The application can be divided into the following components:

Component	Main Function	Important Test Areas
1. Login Module	Authenticates users	Valid and invalid credentials, empty fields, account lockout
2. Session Management	Maintains authenticated sessions	Session expiration, logout, unauthorized access
3. Account Module	Displays account information	Correct balance, account details, access permissions
4. Transaction Module	Processes financial transactions	Valid amount, insufficient balance, transaction limits
5. Balance Update Module	Updates account balances	Accurate deduction, consistency, rollback
6. Transaction History Module	Records completed transactions	Correct amount, timestamp, transaction status
7. Error Handling Module	Handles invalid requests and failures	Clear messages, safe failure, recovery
8. Security Module	Protects user accounts and transactions	Access control, brute-force protection, input validation

This decomposition makes it easier to design focused test cases and identify the component responsible for each defect.

4. Black-Box Testing Techniques

Black-box testing checks application behavior without requiring knowledge of its internal code.

4.1 Equivalence Partitioning

Equivalence Partitioning divides inputs into groups expected to behave similarly. A representative input is selected from each group.

Login test examples:

Input Category	Example	Expected Result
Valid username and password	Correct credentials	Login successful
Valid username and incorrect password	Wrong password	Login rejected
Unknown username	Unregistered username	Login rejected
Empty username	Blank username	Validation message
Empty password	Blank password	Validation message
Both fields empty	No credentials	Validation message

Input Category	Example	Expected Result
Input exceeding permitted length	Excessively long input	Input rejected or safely handled

Transaction test examples:

Input Category	Example	Expected Result
Valid transaction amount	₹500 with sufficient balance	Transaction succeeds
Zero amount	₹0	Transaction rejected
Negative amount	−₹100	Transaction rejected
Amount exceeding balance	₹5,000 with ₹2,000 balance	Transaction rejected
Amount exceeding transaction limit	Amount above the configured limit	Transaction rejected
Invalid amount format	Alphabetic input	Validation message

The exact permitted input ranges and transaction limits must be taken from the application's requirements.

4.2 Boundary Value Analysis

Boundary Value Analysis focuses on values at, immediately below, and immediately above input limits.

Assume, for illustration, that the transaction amount must be greater than ₹0 and cannot exceed ₹25,000 per transaction.

Test Case	Transaction Amount	Expected Result
TC1	−₹1	Reject
TC2	₹0	Reject
TC3	₹1	Accept if all other conditions are satisfied
TC4	₹24,999	Accept if all other conditions are satisfied
TC5	₹25,000	Accept if all other conditions are satisfied
TC6	₹25,001	Reject

Additional balance-boundary tests should include:

- Balance just below the requested amount.
- Balance exactly equal to the requested amount.
- Balance just above the requested amount.

For example, for a ₹1,000 transaction, test balances of ₹999, ₹1,000, and ₹1,001. These tests help identify incorrect comparison operators and balance-validation errors.

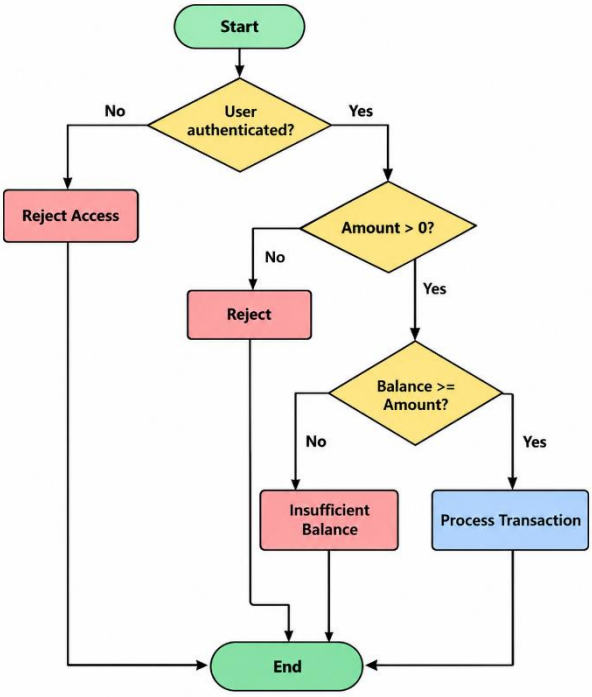
Note: The limits above are illustrative and should be replaced with the application's actual requirements.

5. White-Box Testing Techniques

White-box testing examines internal program logic, conditions, branches, and execution paths.

5.1 Control Flow Graph

A simplified transaction process can be represented as:



This graph illustrates the main decisions in a simplified transaction workflow. A real application may contain additional checks for transaction limits, authorization, database errors, and duplicate requests.

5.2 Decision Points

The main decision points are:

1. Is the user authenticated?
2. Is the transaction amount valid?
3. Is the available balance sufficient?
4. Is the transaction within the permitted limit?
5. Did the transaction processing operation succeed?

Each decision should be tested for all applicable outcomes.

5.3 Independent Execution Paths

Representative paths include:

Path	Execution Sequence	Expected Result
P1	Unauthenticated user → Access rejected	No transaction occurs
P2	Authenticated → Invalid amount → Rejected	Balance unchanged
P3	Authenticated → Valid amount → Insufficient balance	Transaction rejected
P4	Authenticated → Valid amount → Sufficient balance → Processing succeeds	Balance updated correctly
P5	Authenticated → Valid amount → Sufficient balance → Processing fails	No incorrect balance deduction
P6	Authenticated → Amount exceeds transaction limit → Rejected	Transaction not processed

These paths provide a starting point for basis path testing. The complete set of independent paths and the exact cyclomatic complexity must be calculated from the actual source code or control flow graph.

5.4 Statement and Branch Coverage

Statement Coverage: Checks whether executable statements have run at least once.

Branch Coverage: Checks whether each decision outcome, such as True and False, has been executed.

Statement coverage alone is insufficient because a test may execute a statement without exercising every outcome of a decision. Branch coverage provides stronger evidence, although neither criterion guarantees that all possible execution paths or security defects have been tested.

6. Identifying Testing Coverage Gaps

The following gaps may exist if only limited test cases were executed:

Coverage Gap	Possible Consequence	Recommended Improvement
Invalid inputs not tested systematically	Application accepts incorrect data	Apply Equivalence Partitioning
Input limits not tested	Errors at minimum or maximum values	Apply Boundary Value Analysis
Only successful login tested	Authentication defects remain undetected	Test valid and invalid credential combinations
Decision outcomes not fully exercised	Hidden branch defects	Apply Branch Coverage
Transaction failure paths not tested	Incorrect balance updates	Test failure handling and rollback
Repeated requests not tested	Duplicate transactions	Test idempotency and retry behavior
Session expiration not tested	Unauthorized access	Test session timeout and logout
Concurrent transactions not tested	Inconsistent account balances	Test simultaneous requests and database consistency

Coverage Gap	Possible Consequence	Recommended Improvement
Transaction history not verified	Incorrect or missing records	Reconcile transaction records with account balances
Security testing omitted	Unauthorized access or account compromise	Perform authorized security testing

7. Reconstructed Comprehensive Testing Strategy

The reconstructed strategy integrates black-box and white-box testing.

Phase 1: Requirement Analysis

Identify requirements for login credentials, session duration, transaction limits, account balances, authorization, and error handling.

Phase 2: Component Testing

Test the login, session, account, transaction, balance update, history, and error-handling modules independently.

Phase 3: Equivalence Partitioning

Divide login and transaction inputs into valid and invalid classes. Select representative values from each class and verify expected results.

Phase 4: Boundary Value Analysis

Test minimum, maximum, and neighboring values for input length, transaction amounts, balances, and session timeouts.

Phase 5: Decision Table Testing

Create combinations of conditions and expected outcomes.

Authenticated	Amount Valid	Sufficient Balance	Expected Outcome
No	Yes	Yes	Reject access
Yes	No	Yes	Reject transaction
Yes	Yes	No	Reject transaction
Yes	Yes	Yes	Process transaction
Yes	Yes	Yes, but processing fails	Handle failure without an incorrect balance update

Additional rules, such as transaction limits and authorization permissions, should be included in the full decision table.

Phase 6: Path Testing

Construct the control flow graph from the source code. Identify decision nodes and loops, calculate cyclomatic complexity, and select a basis set of independent execution paths. Design test cases that execute each selected path.

Phase 7: Integration and Database Testing

Verify that login, session management, transaction processing, balance updates, and transaction history work together correctly. Confirm that failed transactions do not leave partial updates and that concurrent requests preserve account consistency.

Phase 8: Security and Reliability Testing

Test access control, session expiration, repeated login failures, request tampering, duplicate submissions, and safe error handling. Use only authorized test accounts and environments.

Phase 9: Regression Testing

After fixing defects, rerun the affected tests and the full critical transaction suite to ensure that existing functionality remains correct.

8. Sample Integrated Test Suite

Test ID	Test Scenario	Technique	Expected Result
T01	Login with valid credentials	Equivalence Partitioning	Login successful
T02	Login with incorrect password	Negative Testing	Login rejected
T03	Submit empty login fields	Equivalence Partitioning	Validation message
T04	Submit username at length boundary	Boundary Value Analysis	Correct length validation
T05	Transaction amount is zero	Boundary Value Analysis	Transaction rejected
T06	Transaction amount is at the maximum limit	Boundary Value Analysis	Accepted if permitted
T07	Transaction amount exceeds the maximum limit	Boundary Value Analysis	Transaction rejected
T08	Balance equals transaction amount	Boundary Value Analysis	Transaction succeeds if permitted
T09	Balance is less than transaction amount	Boundary Value Analysis	Transaction rejected
T10	Attempt transaction without authentication	Path Testing	Access rejected
T11	Valid transaction succeeds	Path Testing	Correct balance deduction
T12	Transaction processing fails	Path Testing	No incorrect balance deduction

Test ID	Test Scenario	Technique	Expected Result
T13	Submit the same transaction request repeatedly	Robustness Testing	No unintended duplicate transaction
T14	Session expires before a transaction	State/Session Testing	Reauthentication required
T15	Two transactions execute concurrently	Concurrency Testing	Consistent balance and transaction records
T16	Verify transaction history after completion	Integration Testing	Correct transaction record displayed

9. Justification of the Reconstructed Approach

The combined strategy is effective because each technique addresses a different class of defects.

- **Equivalence Partitioning** reduces the number of test cases while covering representative valid and invalid input groups.
- **Boundary Value Analysis** identifies defects near permitted limits, where comparison and validation errors frequently occur.
- **Path Testing** checks internal control flow and decision outcomes that may not be covered by input-based tests alone.
- **Integration Testing** verifies consistency between transaction processing, database updates, and transaction history.
- **Security Testing** helps identify unauthorized access, session-management weaknesses, and transaction manipulation.
- **Regression Testing** confirms that defect fixes do not introduce new failures.

Defect patterns can guide the investigation, but they cannot conclusively prove which techniques were originally used. The reconstructed strategy should therefore be validated against requirements, actual code, execution traces, test results, and available coverage reports.

10. Conclusion

Reverse engineering the testing approach of a web application requires analyzing defect patterns, decomposing the application into testable components, and evaluating existing coverage. Equivalence Partitioning and Boundary Value Analysis provide systematic input testing, while white-box path testing examines internal decisions and execution flow. Combining these techniques with integration, security, concurrency, and regression testing creates a comprehensive strategy for improving the correctness, reliability, and security of login and transaction operations.

4.Reverse Engineering Black-Box Testing for a Legacy Inventory Management System

1. Introduction

A legacy inventory management system maintains stock quantities, processes stock updates, and generates low-stock alerts when inventory falls below a specified threshold. Since the original requirements and test documentation are unavailable, the business logic must be reconstructed from successful and failed transaction logs. By analyzing these logs, testers can identify possible validation rules, input boundaries, equivalence partitions, and missing test scenarios. Black-box testing techniques, particularly Equivalence Partitioning and Boundary Value Analysis, can then be used to develop a systematic test strategy.

2. Analysis of Transaction Logs

Successful and failed transaction logs provide clues about the system's hidden business rules.

Observed Log Pattern	Possible Hidden Business Rule	Testing Implication
Stock increases successfully for a positive quantity	Positive quantities are accepted	Test valid stock additions
Stock update fails for a negative quantity	Negative quantities are prohibited	Test negative inputs
Stock update fails when the quantity is zero	Zero-quantity updates may be prohibited	Verify the zero boundary
Stock decreases successfully when sufficient stock exists	Stock cannot be reduced below the permitted minimum	Test sufficient and insufficient stock
Update fails when requested stock exceeds available quantity	The system validates available stock	Test values around the available quantity
Low-stock alert appears at a particular stock level	An alert threshold is configured	Test below, at, and above the threshold
No alert appears when stock is sufficiently high	Alerts depend on a threshold condition	Verify normal stock levels
Repeated transactions produce unexpected quantities	Duplicate processing may not be handled correctly	Test repeated requests
Failed updates leave unexpected stock values	Rollback or transaction handling may be defective	Verify stock consistency after failures

Important: These are possible interpretations, not confirmed rules. The actual business logic must be verified against the transaction logs, observed results, and any available system requirements.

3. Decomposition into Functional Units

The inventory update process can be divided into the following functional units.

Functional Unit	Responsibility	Important Test Areas
1. Input Validation	Validates product ID and quantity	Missing values, invalid formats, zero, negative values
2. Product Verification	Checks whether the product exists	Valid, invalid, and inactive product IDs
3. Stock Addition	Increases the current stock	Positive quantities, maximum stock limits
4. Stock Deduction	Decreases the current stock	Sufficient stock, exact stock, insufficient stock
5. Stock Calculation	Calculates the new inventory quantity	Correct addition and subtraction
6. Low-Stock Alert	Determines whether an alert is required	Stock below, equal to, and above the threshold
7. Transaction Logging	Records successful and failed updates	Correct status, quantity, timestamp, and reason
8. Error Handling	Handles rejected updates	No unintended stock change after failure

This decomposition helps testers identify which functions require additional testing and where failures may occur.

4. Equivalence Partitioning

Equivalence Partitioning divides possible inputs into groups that are expected to produce similar results. Instead of testing every possible quantity, representative values are selected from each group.

Assume the system accepts positive whole-number quantities, does not permit negative stock, and requires the product to exist. These assumptions must be confirmed against the actual application.

4.1 Quantity Equivalence Classes

Partition	Example Input	Expected Result
Valid positive quantity	10	Accept the update if other conditions are satisfied
Zero quantity	0	Reject if zero-quantity updates are prohibited
Negative quantity	-5	Reject
Decimal quantity	2.5	Reject if only whole units are supported
Alphabetic quantity	"ABC"	Reject
Missing quantity	Blank	Reject

Partition	Example Input	Expected Result
Quantity exceeding the permitted limit	Above maximum	Reject
Valid quantity that exceeds available stock during deduction	Deduct 20 when only 15 are available	Reject

4.2 Product ID Equivalence Classes

Partition	Example	Expected Result
Existing active product	Valid product ID	Continue processing
Nonexistent product	Unknown ID	Reject
Missing product ID	Blank	Reject
Invalid product ID format	Unsupported characters or format	Reject
Inactive product	Deactivated product ID	Follow the configured business rule

Justification: These partitions cover valid inputs, invalid inputs, and conditions that may prevent an otherwise valid transaction from succeeding.

5. Boundary Value Analysis

Boundary Value Analysis tests values at and near important limits. It is particularly useful for detecting incorrect comparison operators and off-by-one errors.

5.1 Stock Quantity Boundaries

Assume the minimum permitted stock is zero and quantities are whole numbers.

Test Case	Initial Stock	Operation	Quantity	Expected Result
TC1	20	Deduct	19	Success; stock becomes 1
TC2	20	Deduct	20	Success; stock becomes 0 if zero stock is permitted
TC3	20	Deduct	21	Reject; stock remains 20
TC4	20	Add	1	Success; stock becomes 21
TC5	0	Deduct	1	Reject; stock remains 0
TC6	0	Add	1	Success; stock becomes 1
TC7	20	Add	0	Reject if zero-quantity updates are prohibited
TC8	20	Deduct	0	Reject if zero-quantity updates are prohibited

These tests check whether the system handles zero stock, exact stock depletion, and attempts to deduct more stock than is available.

5.2 Low-Stock Alert Boundary

Assume the low-stock threshold is **10 units** and an alert is expected whenever stock is below or equal to the threshold. This is an illustrative assumption.

Test Case	Remaining Stock	Expected Alert
TC9	9	Yes
TC10	10	Yes
TC11	11	No
TC12	0	Yes
TC13	15	No

If the actual business rule is that alerts occur only when stock is strictly below the threshold, the expected result for 10 units must change.

Justification: Testing 9, 10, and 11 helps determine whether the hidden rule uses <, <=, or an incorrect comparison.

5.3 Maximum Quantity Boundary

If the system has a maximum permitted quantity of 1,000 units per update, test:

Test Case	Quantity	Expected Result
TC14	999	Accept if other conditions are satisfied
TC15	1,000	Accept if the limit is inclusive
TC16	1,001	Reject

The actual maximum must be determined from the application's requirements or logs. It should not be assumed to be 1,000 without supporting evidence.

6. Reconstructing the Business Logic

The following simplified logic can be inferred as a starting point:

1. Receive the product ID, operation type, and quantity.
2. Validate that the product exists.
3. Validate that the quantity is in an accepted format and range.
4. For a stock addition, calculate the new quantity by adding the requested amount.
5. For a stock deduction, verify that sufficient stock is available.
6. Reject the transaction if it would violate inventory rules.
7. Save the new stock quantity when the transaction succeeds.
8. Compare the resulting stock level with the configured alert threshold.
9. Generate a low-stock alert when the alert condition is satisfied.
10. Record the transaction result in the logs.

The exact order of validation, stock update, alert generation, and logging should be verified against observed behavior.

7. Identifying Gaps in Existing Testing

The available logs may demonstrate that some scenarios have been tested, but they do not necessarily establish complete coverage.

Possible Gap	Risk	Recommended Test
Only successful updates appear in the logs	Invalid inputs may not be tested	Test zero, negative, missing, and malformed quantities
Only ordinary quantities are tested	Boundary defects may remain hidden	Test values around minimum and maximum limits
Exact stock depletion is not tested	Incorrect rejection or negative stock	Deduct exactly the available quantity
Low-stock threshold is not tested directly	Alerts may trigger too early or too late	Test threshold minus one, threshold, and threshold plus one
Failed transactions are not followed by stock verification	Partial updates may go undetected	Verify stock remains unchanged after rejection
Unknown product IDs are not tested	Invalid records may be created	Test nonexistent and inactive products
Repeated updates are not tested	Duplicate processing may alter stock incorrectly	Repeat the same request and inspect the resulting stock
Concurrent updates are not tested	Stock may become inconsistent	Test simultaneous deductions against the same inventory
Alert delivery is not verified	Alerts may be missing or duplicated	Check alert generation and delivery records
Logs are not reconciled with stock values	Audit information may be inaccurate	Compare transaction logs with final inventory quantities

8. Reconstructed Black-Box Test Design

The following integrated test suite combines Equivalence Partitioning, Boundary Value Analysis, and functional testing.

For the examples, assume an initial stock of 20 units, whole-number quantities, no negative inventory, and a low-stock alert threshold of 10 units. Assume the alert triggers at or below the threshold.

Test ID	Scenario	Input or Operation	Technique	Expected Result
BT01	Add valid stock	Add 5 to 20	Equivalence Partitioning	Stock becomes 25
BT02	Deduct valid stock	Deduct 5 from 20	Equivalence Partitioning	Stock becomes 15
BT03	Deduct exact available stock	Deduct 20 from 20	Boundary Value Analysis	Stock becomes 0

Test ID	Scenario	Input or Operation	Technique	Expected Result
BT04	Deduct more than available	Deduct 21 from 20	Boundary Value Analysis	Rejected; stock remains 20
BT05	Add zero quantity	Add 0	Equivalence Partitioning	Rejected if zero updates are prohibited
BT06	Add negative quantity	Add -5	Equivalence Partitioning	Rejected
BT07	Use malformed quantity	Enter "ABC"	Equivalence Partitioning	Rejected
BT08	Use missing product ID	Leave product ID blank	Negative Testing	Rejected
BT09	Use unknown product ID	Enter nonexistent ID	Equivalence Partitioning	Rejected
BT10	Resulting stock below threshold	Reduce stock to 9	Boundary Value Analysis	Alert generated
BT11	Resulting stock at threshold	Reduce stock to 10	Boundary Value Analysis	Alert generated
BT12	Resulting stock above threshold	Set stock to 11	Boundary Value Analysis	No alert
BT13	Exceed maximum update quantity	Submit maximum plus one	Boundary Value Analysis	Rejected
BT14	Repeat the same update request	Submit an identical request twice	Robustness Testing	Behavior follows retry policy; no unintended duplicate processing
BT15	Verify stock after rejected deduction	Deduct more than available	State Verification	Stock remains unchanged
BT16	Perform concurrent deductions	Two requests use the same available stock	Concurrency Testing	No negative or inconsistent stock
BT17	Verify transaction history	Perform a successful update	Integration Testing	Correct transaction is recorded
BT18	Verify failed transaction log	Attempt an invalid update	Log Verification	Failure reason is recorded accurately

9. Justification of the Selected Test Cases

The reconstructed test cases are selected for the following reasons:

- **Equivalence Partitioning** covers representative valid and invalid inputs without requiring every possible input value to be tested.
- **Boundary Value Analysis** focuses on stock limits and alert thresholds, where comparison and calculation errors are likely.
- **Functional Testing** verifies that stock addition and deduction produce the expected inventory quantities.
- **Negative Testing** checks that invalid inputs and prohibited operations are rejected safely.
- **State Verification** confirms that a failed transaction does not modify stock incorrectly.
- **Integration Testing** checks whether stock updates, alerts, and transaction logs remain consistent.

- **Concurrency Testing** helps identify inconsistent results when multiple transactions update the same product.
- **Log Verification** helps determine whether recorded outcomes match the actual inventory state.

The expected outcomes should be confirmed against documented requirements or carefully validated observations from the legacy system.

10. Conclusion

Reverse engineering a legacy inventory management system requires analyzing transaction logs to infer hidden validation rules, stock boundaries, and low-stock alert conditions. Decomposing the system into functional units makes it easier to identify untested areas. Equivalence Partitioning covers representative input categories, while Boundary Value Analysis tests values around stock limits and alert thresholds. Integrating these techniques with functional, negative, concurrency, and log-verification testing provides a systematic black-box strategy for identifying defects and improving inventory accuracy and reliability.

5.White-Box Testing of a Legacy Library Management System Using CPU and Memory Profiles

1. Introduction

A compiled .exe file of a library management system contains executable logic, but its original source code and testing documentation may not be available. CPU and memory profiles collected during stress testing can help identify operations that consume excessive resources. Spikes in CPU usage may indicate repeated loops, complex calculations, or frequently executed decision branches. Memory growth may indicate large data structures, repeated allocations, or possible memory leaks. By correlating these observations with application behavior, a conceptual Control Flow Graph (CFG) and a targeted path-testing strategy can be developed to evaluate performance and system stability.

Limitation: CPU and memory profiles alone cannot reveal the exact source code or prove which loops and branches exist. The analysis below presents a conceptual model; actual conclusions require the executable, profiling data, and, where possible, debugging symbols or disassembly.

2. Analysis of CPU and Memory Execution Profiles

During stress testing, the following patterns may indicate potential problems.

Observed Profile Pattern	Possible Cause	Recommended Investigation
Sudden CPU usage spike during book search	Repeated search iterations or inefficient searching	Examine search behavior with small and large catalogues
CPU usage increases with the number of issued books	Nested loops or repeated processing of issue records	Test borrowing and issue-history operations
Memory usage rises continuously during repeated searches	Memory leak or retained search results	Monitor memory across repeated operations

Observed Profile Pattern	Possible Cause	Recommended Investigation
Memory spikes when loading all books	Large collections or unnecessary object allocation	Test catalogue loading with increasing data sizes
CPU remains high after a request finishes	Background processing, excessive polling, or an unintended loop	Examine idle behavior and background tasks
Application slows when several users borrow books simultaneously	Lock contention, database delays, or shared-resource conflicts	Conduct concurrent-user testing
CPU and memory increase together during report generation	Large result sets, sorting, filtering, or report aggregation	Test report generation with different record counts
Application freezes with very large input	Unbounded iteration, excessive resource consumption, or blocking operations	Test large datasets with timeouts and resource monitoring

These patterns help prioritize code paths for investigation. They do not uniquely identify the underlying cause, so each hypothesis must be verified through controlled tests.

3. Decomposition of the Library Management System

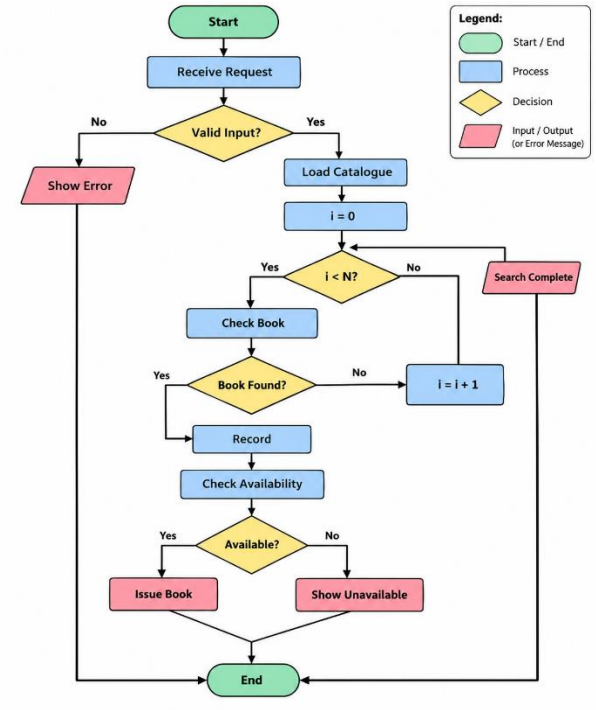
The application can be divided into the following functional units.

Module	Function	Possible Resource-Intensive Operations
1. Book Search	Finds books by title, author, or ID	Iteration over catalogue records
2. Book Registration	Adds new books to the catalogue	Validation and collection updates
3. Borrowing Module	Issues books to registered members	Availability checks and issue-record processing
4. Return Module	Processes returned books	Fine calculations and record updates
5. Member Management	Adds and searches member records	Member-list traversal
6. Fine Calculation	Calculates overdue charges	Date calculations and repeated record processing
7. Report Generation	Produces inventory and transaction reports	Filtering, sorting, aggregation, and memory allocation
8. Database Access	Reads and updates persistent records	Repeated queries and transaction processing

This decomposition helps associate each profile spike with a specific application operation.

4. Conceptual Control Flow Graph (CFG)

Consider a simplified book-search and borrowing operation.



CFG interpretation:

- **Decision nodes:** Input validation, loop condition, book-found condition, and availability condition.
- **Loop:** The catalogue search repeats until a matching book is found or all records have been examined.
- **Resource-intensive path:** Loading a large catalogue and repeatedly checking records can increase CPU usage and memory consumption.
- **Potential defect:** If the loop index is not updated correctly, the search may fail to terminate.
- **Important consideration:** The graph is illustrative and does not establish the actual internal structure of the compiled executable.

5. Identifying Independent Execution Paths

The following paths can be used as an initial path-testing plan.

Path	Execution Sequence	Purpose
P1	Start → Invalid input → Error → End	Test validation and early termination
P2	Start → Valid input → Catalogue loaded → No books found → End	Test empty catalogue handling
P3	Start → Valid input → Search loop → Book found → Available → Issue book → End	Test successful borrowing
P4	Start → Valid input → Search loop → Book found → Unavailable → End	Test unavailable-book handling
P5	Start → Valid input → Search loop → No match after all records → End	Test unsuccessful search
P6	Start → Valid input → Search loop → Large catalogue → Search completion	Evaluate performance at scale
P7	Start → Valid input → Search loop → Processing error → Safe termination	Evaluate error handling and recovery

Paths P6 and P7 represent performance and robustness scenarios that should be added to the functional path tests. The exact set of independent paths and the cyclomatic complexity must be derived from the actual control flow graph.

6. Reconstructing the Path-Testing Strategy

6.1 Functional Path Testing

Execute each identified path with controlled inputs and verify the expected result.

- Submit invalid and valid search requests.
- Search an empty catalogue.
- Search for a book at the beginning, middle, and end of the catalogue.
- Search for a nonexistent book.
- Attempt to borrow an available book.
- Attempt to borrow an unavailable book.
- Test behavior when an error occurs during a search or issue operation.

These cases verify functional correctness before performance measurements are interpreted.

6.2 Loop Testing

Loops are important because their execution counts often determine resource usage.

For a catalogue-search loop, test:

Test Case	Catalogue Size	Search Condition	Purpose
L1	0 records	Empty catalogue	Verify safe handling of zero iterations
L2	1 record	Matching book	Test minimum nonempty catalogue

Test Case	Catalogue Size	Search Condition	Purpose
L3	10 records	Match at first record	Test early loop termination
L4	10 records	Match at final record	Test near-maximum iteration count
L5	10 records	No matching book	Test complete loop execution
L6	10,000 records	Match near the end	Examine large-input performance
L7	10,000 records	No matching book	Examine worst-case search behavior

The large catalogue sizes are illustrative and should be adjusted to the application's expected workload and available resources.

6.3 Decision and Branch Testing

Every identified decision should be exercised through its relevant outcomes.

For example:

- Valid input versus invalid input.
- Book found versus book not found.
- Available book versus unavailable book.
- Successful processing versus processing failure.

Branch coverage can reveal untested decision outcomes, but it does not guarantee that every possible execution path has been tested.

6.4 Stress and Resource Testing

Run the same functional paths under increasing workloads.

1. Establish a baseline with a small catalogue and one user.
2. Increase the catalogue size gradually.
3. Repeat searches and borrowing operations.
4. Increase the number of concurrent users.
5. Monitor CPU utilization, memory usage, response time, and error rate.
6. Repeat the test to determine whether memory returns to a stable level after operations finish.
7. Stop testing if the application exceeds defined safety limits.

Record the input size, operation, number of iterations, CPU usage, peak memory, response time, and outcome for every run.

7. Evaluating Cyclomatic Complexity

Cyclomatic complexity estimates the number of linearly independent paths through a control flow graph.

The formula is:

$V(G) = E - N + 2P$

Where:

- **E** = Number of control-flow edges.
- **N** = Number of nodes.
- **P** = Number of connected components, normally 1 for a single routine.

For a single connected graph, another commonly used formula is:

V(G) = Number of decision nodes + 1

In the illustrative CFG, there are four decision nodes:

1. Input validation.
2. Loop condition.
3. Book-found condition.
4. Availability condition.

Therefore, the illustrative cyclomatic complexity is:

V(G) = 4 + 1 = 5

This suggests five independent paths for the simplified graph. It does not mean that only five tests are needed: loop boundaries, large inputs, error handling, concurrency, and resource behavior require additional test cases.

The actual executable's complexity cannot be calculated reliably without reconstructing its real control flow.

8. Evaluating System Stability

The impact of the reconstructed testing strategy can be evaluated using the following criteria.

Metric	Potential Problem	Desired Outcome
CPU utilization	Excessive processing or inefficient loops	CPU use remains within defined limits
Memory usage	Leaks or unnecessary allocations	Memory stabilizes after repeated operations
Response time	Slow searches or reports	Response time meets requirements
Error rate	Failures under heavy workloads	Error rate remains within acceptable limits
Search correctness	Missing or incorrect results	Results remain correct for all tested catalogue sizes
Data consistency	Duplicate issues or incorrect availability	Book status and issue records remain consistent
Recovery	Application remains unusable after a failure	Application recovers safely or terminates cleanly

Stability Assessment

A test result should be classified as successful only when the application produces the correct result and stays within the defined performance and resource limits.

For example, a search that returns the correct book but causes memory usage to grow continuously should be considered a stability defect. Similarly, a fast search that returns incorrect results is a functional failure.

9. Limitations and Recommended Tools

A compiled executable does not automatically provide access to source-level control flow. The analysis may require:

- A debugger or disassembler to inspect executable instructions.
- A CPU profiler to identify expensive functions or instruction regions.
- A memory profiler to track allocations and possible leaks.
- Application logs to correlate spikes with user operations.
- Controlled tests with repeatable inputs and workloads.

The specific tools depend on the executable's platform, runtime, symbols, and available profiling information. If only CPU and memory graphs are available, the CFG remains a hypothesis rather than a verified representation of the program.

10. Conclusion

Reverse engineering white-box testing requirements from a compiled library management application involves correlating CPU and memory spikes with likely loops, decision clusters, and resource-intensive operations. A conceptual CFG helps organize the suspected execution paths, while branch, path, loop, and stress testing evaluate correctness and stability. By monitoring resource usage alongside functional results, testers can identify inefficient processing, excessive memory consumption, and failures that occur only under heavy workloads. However, a reliable white-box assessment requires validating the conceptual model against actual executable behavior and available profiling evidence.

6.State Transition Testing of a Mobile Payment Gateway API

1. Introduction

A mobile payment gateway API processes transactions through different states, such as Pending, Success, Reversal, and Timeout. Each state represents the current status of a payment, while API responses provide information about transaction progress and outcomes. By analyzing response status codes, transaction IDs, timestamps, and other data constraints, testers can reconstruct the underlying state machine. **State Transition Testing** is the primary black-box testing technique used to verify valid transitions, reject illegal transitions, and ensure that transaction states remain consistent.

Note: The actual JSON responses were not included in the question. Therefore, the state machine and test cases below represent a reasonable example. The exact transition rules must be confirmed against the API responses and payment gateway requirements.

2. Identification of the Testing Technique

The implicit testing technique is **State Transition Testing**.

This technique is appropriate because the expected API response depends on the transaction's current state and the event that occurs.

For example:

- A Pending transaction may become Success after payment confirmation.
- A Pending transaction may become Reversal if a payment is cancelled or a confirmed payment must be reversed.
- A Pending transaction may become Timeout if no response arrives within the permitted period.
- A terminal transaction may reject subsequent state-changing requests.

State Transition Testing checks both the current state and the event that triggers the next state.

3. Identification of States and Transition Triggers

3.1 Transaction States

State	Meaning	Typical Data Constraints
Pending	Transaction has started but is not yet finalized	Valid transaction ID; final outcome not yet confirmed
Success	Payment has completed successfully	Successful status; valid transaction reference
Reversal	A previously authorized or completed payment has been reversed, according to gateway rules	Reversal reference or related original transaction ID
Timeout	Transaction did not receive a final response within the permitted time	Timeout status; transaction reference retained

The exact meaning of Reversal must be confirmed. Some gateways allow reversal only after authorization, while others use it to compensate for a completed transaction.

3.2 Transition Triggers

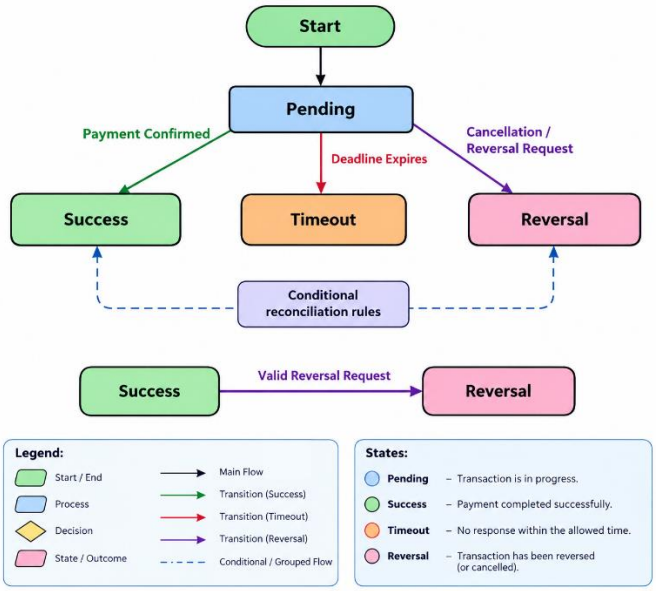
Current State	Trigger	Possible Next State
Pending	Payment confirmation received	Success
Pending	Cancellation or reversal request accepted	Reversal, if permitted by the API
Pending	Response deadline expires	Timeout
Timeout	Late payment confirmation received	Depends on the documented reconciliation policy
Success	Valid reversal request accepted	Reversal

Current State	Trigger	Possible Next State
Reversal	Duplicate reversal request	Remain Reversal or return an appropriate error
Success	Duplicate success notification	Remain Success without duplicate processing
Timeout	Repeated timeout notification	Remain Timeout or return an appropriate response

These transitions are examples. The API specification determines which transitions are legal and how repeated or late events must be handled.

4. Structured State-Transition Diagram

The following conceptual diagram represents the main transaction lifecycle.



Interpretation:

- Pending is the initial processing state.
- Success indicates a completed payment.
- Timeout indicates that the expected response was not received in time.
- Reversal represents a permitted compensating action.
- Transitions from Timeout depend on the gateway's late-response and reconciliation rules.
- A terminal state should not be changed arbitrarily by duplicate or out-of-order requests.

For a formal implementation, each transition should have a defined trigger, guard condition, expected API response, and resulting transaction state.

5. Analysis of JSON Response Constraints

A payment API response may contain fields such as:

```
{
  "transactionId": "TXN1001",
  "status": "Pending",
  "amount": 500,
  "currency": "INR",
  "timestamp": "2026-10-22T10:30:00Z"
}
```

This is an illustrative response, not a supplied API response.

The following fields should be validated:

JSON Field	Validation Requirement
transactionId	Must be present and identify the correct transaction
status	Must contain an allowed state value
amount	Must be numeric and satisfy the gateway's amount rules
currency	Must be supported by the API
timestamp	Must use a valid, documented format
originalTransactionId	Required for a reversal if specified by the API
errorCode	Must be consistent with the outcome
responseCode	Must agree with the transaction status

Important Data Constraints

1. A transaction ID must remain associated with the correct payment.
2. A successful transaction must not be treated as Pending without an explicitly permitted reconciliation process.
3. A reversal must reference the correct original transaction.
4. Duplicate requests must not create duplicate charges or reversals.
5. The amount and currency must remain consistent across related responses.
6. Invalid or missing fields must be handled according to the API contract.
7. Late responses must follow the documented timeout-recovery policy.

6. Reconstructed State-Based Test Suite

6.1 Legal Transition Tests

Test ID	Initial State	Event or Trigger	Expected State	Expected Result
ST01	None	Initiate transaction	Pending	Transaction created
ST02	Pending	Valid payment confirmation	Success	Payment confirmed
ST03	Pending	Deadline expires	Timeout	Timeout recorded
ST04	Pending	Valid cancellation/reversal event, if supported	Reversal	Event handled according to policy
ST05	Success	Valid reversal request	Reversal	Reversal processed once
ST06	Success	Duplicate success notification	Success	No duplicate charge
ST07	Reversal	Duplicate reversal request	Reversal	No duplicate reversal
ST08	Timeout	Late payment confirmation	Policy-defined outcome	Reconciliation handled correctly

6.2 Illegal Transition Tests

Test ID	Current State	Attempted Event	Expected Result
ST09	Reversal	Attempt to process the same payment again as a new success	Reject or ignore according to API rules
ST10	Reversal	Request a second reversal	Reject or return an idempotent response
ST11	Success	Submit an invalid state-changing request	Reject without changing state
ST12	Pending	Submit an unsupported status value	Reject or safely ignore
ST13	Any state	Submit a request with a missing transaction ID	Validation error
ST14	Any state	Submit a request with a mismatched transaction ID	Reject; do not alter another transaction
ST15	Success	Submit a reversal for a different transaction	Reject
ST16	Timeout	Replay a stale or duplicate callback	Follow idempotency and reconciliation rules

Illegal-transition tests must be based on documented rules. A transition should not be labelled illegal solely because it is uncommon.

7. Transition Coverage

State-transition coverage can be evaluated at different levels.

7.1 State Coverage

Every defined state must be visited at least once:

- Pending
- Success
- Reversal
- Timeout

7.2 Transition Coverage

Every documented legal transition must be executed at least once.

For example:

- Pending → Success
- Pending → Timeout
- Success → Reversal

Any additional transition permitted by the API must also be tested.

7.3 Invalid Transition Coverage

Attempt each identified illegal transition and verify that the API rejects or safely handles it without corrupting the transaction state.

7.4 Transition-Pair Coverage

Test pairs of consecutive transitions, where applicable. For example:

Pending → Success → Reversal

This checks that a reversal works correctly after successful payment processing.

Another sequence is:

Pending → Timeout → Late Confirmation

This checks the gateway's reconciliation policy.

8. API Validation and Negative Testing

In addition to state transitions, test the following:

- Malformed JSON.
- Missing required fields.
- Invalid status values.
- Invalid amount formats.
- Negative or zero amounts where prohibited.
- Unsupported currency values.
- Invalid timestamps.
- Duplicate transaction requests.
- Duplicate callbacks.
- Out-of-order callbacks.
- Network interruption during payment processing.
- Delayed responses and timeout handling.
- Concurrent requests affecting the same transaction.

For every rejected request, verify that the transaction state, balance, and transaction history remain consistent.

9. Evaluating Test Adequacy

Coverage Area	Possible Gap	Recommended Improvement
State coverage	One or more states never tested	Test all four states
Transition coverage	Only successful payment tested	Test timeout and reversal transitions
Invalid transition coverage	Illegal state changes not attempted	Add negative transition tests
Data validation	JSON fields not checked thoroughly	Validate required fields and data types
Duplicate handling	Repeated requests not tested	Test idempotency
Timeout recovery	Late responses not examined	Test delayed and out-of-order callbacks
Reversal integrity	Original transaction not verified	Check transaction references and amounts
Concurrency	Simultaneous callbacks not tested	Test duplicate and competing events
Regression	Fixes may break existing transitions	Rerun the full state-transition suite

10. Conclusion

State Transition Testing is the most appropriate technique for reconstructing the behavior of a mobile payment gateway from its JSON responses. By identifying transaction states, transition triggers, data constraints, and legal and illegal transitions, testers can develop a comprehensive state-based test suite. State coverage, transition coverage, transition-pair coverage, and invalid-transition testing help detect inconsistent transaction statuses, duplicate processing, and incorrect timeout or reversal handling. The final state machine should be validated against the actual API responses and documented business rules to ensure payment correctness, reliability, and consistency.

7.Black-Box Fuzzing and Boundary Value Analysis of an Encrypted Communication Module

1. Introduction

An encrypted communication module accepts input parameters, validates them, encrypts the accepted data, and produces encrypted outputs. When the internal source code is unavailable, black-box testing can be used to examine how the module responds to valid, invalid, and unexpected inputs. Fuzzing helps identify unexpected failures by supplying malformed or unusual input values, while Boundary Value Analysis (BVA) focuses on values near input limits. Combining these techniques helps identify validation weaknesses, prevent unsafe memory operations, and improve the reliability and security of the communication module.

2. Reverse Engineering Through Fuzzing Analysis

Fuzzing is a software testing technique in which a program is supplied with varied inputs to discover crashes, unexpected responses, validation failures, or inconsistent behavior.

For the encrypted communication module, testers can vary input length, data type, encoding, special characters, and cryptographic parameters.

2.1 Input Fields and Possible Validation Rules

Input Field	Fuzzing Variation	Expected Behavior
Message	Empty string	Accept or reject according to the specification
Message	Extremely long input	Reject safely if the permitted size is exceeded
Message	Special characters and symbols	Handle safely without corrupting processing
Message	Unicode characters	Accept supported encodings or return a clear validation error
Message length	Values around the maximum limit	Correctly enforce the length restriction
Key	Missing or invalid key	Reject without attempting unsafe encryption
Key length	Below or above the permitted size	Reject unsupported key lengths
Initialization vector or nonce	Missing or incorrectly sized value	Reject if required by the selected algorithm
Encoding	Invalid hexadecimal or Base64 input	Return a controlled decoding error
Metadata	Unexpected or malformed fields	Reject or safely handle according to the input schema

The exact validation rules depend on the cryptographic algorithm, API contract, and supported input format.

2.2 Interpreting Fuzzing Results

Observed Behavior	Possible Defect	Recommended Investigation
Application crashes on long input	Unsafe buffer handling or resource exhaustion	Test input-size boundaries and inspect memory-safety diagnostics
Application hangs on malformed input	Inefficient parsing or uncontrolled processing	Add timeouts and monitor resource usage
Invalid keys are accepted	Inadequate cryptographic parameter validation	Verify key-format and key-length checks
Special characters change the structure of a request	Incorrect parsing or input injection	Test field separation and structured-input validation
Invalid encoding causes an unhandled exception	Missing error handling	Test malformed encoding and verify controlled rejection
Repeated requests produce inconsistent results	State-management or concurrency defect	Repeat tests and compare results
Error responses reveal sensitive information	Information leakage	Check that errors do not expose keys, plaintext, or internal details

Important: A crash or unexpected response does not by itself prove a buffer overflow or injection vulnerability. The finding must be confirmed through appropriate diagnostics and controlled testing.

3. Decomposition into Functional Components

The module can be divided into the following testable components.

Component	Responsibility	Main Testing Focus
1. Input Receiver	Receives message and encryption parameters	Missing fields, invalid data types, oversized inputs
2. Data Validation	Checks formats, lengths, and required fields	Boundary values, malformed data, unsupported encodings
3. Input Decoder	Converts encoded data into its internal representation	Invalid Base64, malformed hexadecimal, Unicode handling
4. Key Validation	Checks cryptographic key parameters	Incorrect length, unsupported key format, missing key
5. Encryption Engine	Encrypts validated plaintext	Correct operation, valid parameters, error handling
6. Output Encoder	Converts ciphertext into the required output format	Output length, encoding validity, truncation
7. Error Handler	Reports invalid inputs and processing failures	Controlled errors, no crashes, no sensitive information leakage
8. Resource Manager	Manages buffers and temporary data	Memory safety, resource limits, cleanup after failure

This decomposition allows testers to distinguish input-validation failures from encryption failures and output-handling defects.

4. Boundary Value Analysis (BVA)

Boundary Value Analysis concentrates on values at and near the permitted input limits.

Assume, **for illustration only**, that the API accepts message lengths from 1 to 1,024 bytes. The actual limits must be obtained from the module's specification or established through controlled testing.

4.1 Message-Length Boundary Tests

Test ID	Input Length	Test Purpose	Expected Result
BVA01	0 bytes	Test the minimum boundary	Accept or reject according to the specification
BVA02	1 byte	Test the minimum valid length	Accept
BVA03	2 bytes	Test just above the minimum	Accept
BVA04	1,023 bytes	Test just below the maximum	Accept
BVA05	1,024 bytes	Test the maximum permitted length	Accept
BVA06	1,025 bytes	Test just above the maximum	Reject safely
BVA07	Significantly larger than the maximum	Test oversized-input handling	Reject safely without excessive resource consumption

The empty-input test is included because some encryption APIs permit empty plaintext while others prohibit it. Its expected result must follow the documented requirements.

4.2 Key-Length Boundary Tests

Key-length validation must follow the selected cryptographic algorithm.

For example, if the module uses AES, valid AES key sizes are 128, 192, and 256 bits.

Test ID	Key Input	Expected Result
BVA08	Missing key	Reject
BVA09	127-bit key	Reject if AES key validation is performed on the decoded key
BVA10	128-bit key	Accept for AES
BVA11	192-bit key	Accept for AES
BVA12	256-bit key	Accept for AES
BVA13	257-bit key	Reject as an unsupported AES key size

These examples assume that the key has already been decoded into its actual binary representation. If the API accepts encoded key strings, the encoded input length and decoded key length must be tested separately.

4.3 Encoded-Input Boundary Tests

Test ID	Input	Expected Result
BVA14	Empty encoded string	Handle according to the specification
BVA15	Valid encoded input	Decode successfully
BVA16	Invalid encoding characters	Reject with a controlled error
BVA17	Truncated encoded input	Reject if incomplete data is not permitted
BVA18	Encoded input exceeding the maximum size	Reject safely
BVA19	Input containing unexpected padding	Accept only if permitted by the encoding rules

These tests help identify decoder errors, length-calculation mistakes, and unsafe handling of malformed data.

5. Improved Fuzzing Test Cases

After identifying the expected boundaries, fuzzing can be used to test combinations that ordinary boundary tests may miss.

Test ID	Fuzzing Scenario	Expected Result
FZ01	Random valid-length messages	Controlled processing
FZ02	Messages containing null bytes	Safe handling without unintended truncation
FZ03	Messages containing Unicode characters	Correct encoding or controlled rejection
FZ04	Very long messages	Enforced size limits
FZ05	Random malformed encoded strings	Controlled decoding errors
FZ06	Missing required fields	Validation error
FZ07	Incorrect field data types	Schema validation failure
FZ08	Repeated malformed requests	No crash or uncontrolled resource growth
FZ09	Invalid cryptographic parameters	Rejection before encryption
FZ10	Multiple concurrent valid requests	Correct, isolated processing
FZ11	Inputs containing delimiters and control characters	No unintended field or command interpretation
FZ12	Interrupted processing	Safe cleanup and consistent error handling

Fuzzing should be performed in an authorized test environment using synthetic data and non-production keys.

6. Preventing Buffer Overflow Vulnerabilities

A buffer overflow can occur when a program writes more data into a memory buffer than the buffer can hold.

The following design and testing measures help reduce this risk:

1. **Input-length validation:** Reject inputs exceeding the documented maximum before copying or processing them.

2. **Safe memory allocation:** Allocate buffers based on validated lengths and the encryption algorithm's documented output-size requirements.
3. **Integer-overflow checks:** Verify size calculations before allocating memory or computing output-buffer capacity.
4. **Bounds checking:** Ensure that read and write operations remain within allocated memory.
5. **Safe library usage:** Prefer memory-safe language features and well-maintained cryptographic libraries over custom memory-management code.
6. **Controlled failure handling:** Ensure invalid inputs do not cause crashes, unbounded retries, or excessive memory consumption.
7. **Memory-safety testing:** Use suitable runtime diagnostics, sanitizers, or platform-specific analysis tools when source code is available.

For example, if the API accepts at most 1,024 bytes, an input of 1,025 bytes should be rejected before an unsafe copy occurs.

Limitation: Black-box testing can reveal suspicious behavior, but it cannot guarantee that a buffer overflow is absent. Source-level analysis and memory-safety tools provide stronger evidence.

7. Preventing Injection Vulnerabilities

Injection vulnerabilities arise when untrusted input is interpreted as commands, query syntax, or unintended control information.

Although an encryption module may not directly execute commands, its input parser, metadata handler, or surrounding API may still be vulnerable.

Recommended measures include:

- Validate inputs against a clearly defined schema.
- Use structured data parsers rather than manual string concatenation.
- Keep message content separate from command or metadata fields.
- Avoid passing user-controlled input directly to shell commands or database queries.
- Use parameterized database queries if the module interacts with a database.
- Reject malformed fields and unsupported encodings.
- Apply appropriate output encoding when displaying error messages or logs.
- Ensure error responses do not expose plaintext, cryptographic keys, or sensitive internal details.

Fuzzing with special characters and malformed fields helps identify parsing weaknesses, but actual injection vulnerabilities must be confirmed in the relevant application context.

8. Encryption and Output Validation

The encryption component should be tested independently from input validation.

Important checks include:

1. Valid plaintext produces ciphertext in the documented output format.
2. Decryption by the corresponding authorized process recovers the original plaintext.
3. Modified ciphertext is rejected when authenticated encryption is used.
4. Invalid keys and required cryptographic parameters are rejected.

5. Encryption failures do not return partially processed or misleading results.
6. Repeated encryption behaves according to the selected algorithm and mode. For randomized or nonce-based encryption, different ciphertexts may be expected for the same plaintext.
7. Keys and plaintext are not unintentionally exposed through logs or error responses.

Testing should use standard cryptographic libraries and test vectors appropriate to the chosen algorithm rather than relying on custom encryption implementations.

9. Evaluating the Improved Test Design

Testing Technique	Main Benefit	Defects It May Help Identify
Boundary Value Analysis	Tests minimum and maximum input limits	Off-by-one errors and incorrect length validation
Equivalence Partitioning	Covers representative input categories	Missing, malformed, and unsupported inputs
Fuzzing	Explores unexpected input combinations	Crashes, parser errors, and abnormal resource usage
Negative Testing	Checks rejection of invalid inputs	Inadequate validation and error handling
Memory-Safety Testing	Examines unsafe memory operations	Buffer overflows and related memory errors
Integration Testing	Tests validation, encryption, and output handling together	Incorrect data flow between components
Regression Testing	Repeats previous tests after fixes	Reintroduced validation or processing defects

The improved strategy combines predictable boundary tests with less predictable fuzzing inputs. This provides broader input coverage and helps detect both specification violations and unexpected failures.

10. Conclusion

Black-box fuzzing and Boundary Value Analysis can be used to reverse-engineer the validation behavior of an encrypted communication module. By decomposing the system into input validation, decoding, key validation, encryption, output encoding, and error handling, testers can identify important test boundaries and missing scenarios. BVA verifies values around permitted limits, while fuzzing explores malformed and unusual input combinations. Together with memory-safety analysis, secure parsing, and cryptographic-library validation, these techniques help reduce the risk of buffer overflows, injection vulnerabilities, and unexpected application failures.

8.White-Box Path Testing of an Insurance Premium Calculator

1. Introduction

An insurance premium calculator determines the premium payable by a customer based on factors such as age, vehicle type, insurance coverage, and driving history. Legacy unit tests report 100% statement coverage but fail to identify defects in multi-condition if statements. This indicates that the existing tests execute all statements but may not exercise every decision outcome or combination of Boolean conditions. White-box techniques such as Control Flow Graph (CFG) analysis, Branch Coverage, Condition Coverage, and Path Testing can help identify these gaps and improve calculation accuracy.

2. Reverse Engineering the Original Testing Approach

The original test suite appears to have used **Statement Coverage**.

Statement Coverage measures whether every executable statement has been executed at least once.

Formula:

Statement Coverage=Executed StatementsTotal Executable Statements×100\text{Statement Coverage} = \frac{\text{Executed Statements}}{\text{Total Executable Statements}} \times 100

Although the tests achieved 100% statement coverage, they failed to detect defects in compound conditions.

Example of a Multi-Condition Statement

Consider the following illustrative logic:

```
if (age < 25 && accidents > 0) {
   premium = premium + 2000;
}
```

The statement inside the if may execute during testing, but the existing tests might not adequately exercise the different outcomes of both conditions.

For example:

Test	Age < 25	Accidents > 0	Overall Condition
TC1	True	True	True
TC2	False	False	False

These tests exercise the overall condition as True and False, but they do not isolate the effect of each individual condition.

Inadequacy: Statement Coverage does not guarantee Branch Coverage, Condition Coverage, or complete Path Coverage.

3. Proposed Insurance Premium Logic

To reconstruct the testing approach, assume the calculator uses the following simplified rules:

1. If the customer is under 25 and has previous accidents, an additional risk premium is charged.
2. If the customer has selected comprehensive insurance, an additional coverage premium is charged.
3. Otherwise, the base premium is calculated without the corresponding surcharge.

The following example code represents these assumptions:

```
if (age < 25 && accidents > 0) {
   premium += 2000;
}

if (comprehensiveInsurance) {
   premium += 3000;
}
```

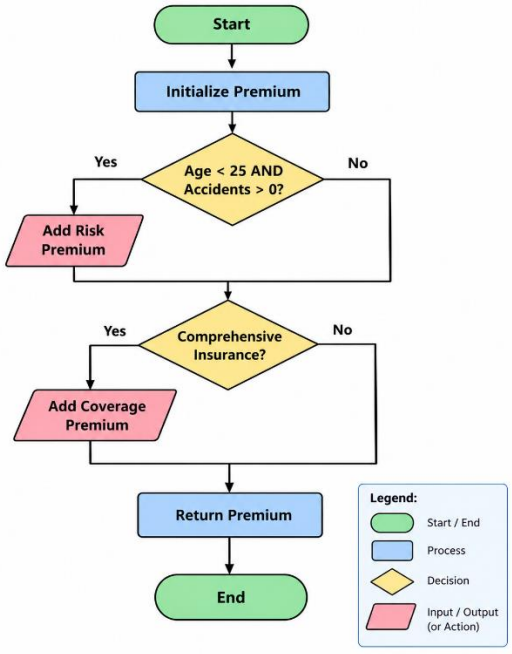
Here:

- age < 25 checks the customer's age.
- accidents > 0 checks whether the customer has previous accidents.
- comprehensiveInsurance indicates whether comprehensive coverage was selected.

These are illustrative rules; an actual insurance calculator may use different conditions and premium amounts.

4. Control Flow Graph (CFG)

The CFG represents the flow of control through the program, including decision points and execution paths.



Decision Points

The graph contains two major decision nodes:

- **D1:** age < 25 && accidents > 0
- **D2:** comprehensiveInsurance

The first decision contains two individual Boolean conditions, while the second is a single Boolean condition.

Each decision has True and False outcomes that must be tested.

5. Cyclomatic Complexity

Cyclomatic Complexity estimates the number of linearly independent paths through the control flow graph.

The formula is:

$$V(G)=E-N+2P$$

Where:

- EE = Number of edges.
- NN = Number of nodes.
- PP = Number of connected components, normally 1 for a single routine.

For a single connected graph, a common alternative is:

$$V(G)=\text{Number of decision nodes}+1$$

There are two decision nodes, so:

$$V(G)=2+1=\boxed{3}$$

Therefore, the simplified CFG has **three linearly independent paths**.

This does not mean that three tests will necessarily achieve every coverage criterion. The compound condition must also be tested at the individual-condition level.

6. Independent Execution Paths

Path	Risk Condition	Comprehensive Coverage	Expected Execution
P1	True	True	Add risk premium and coverage premium
P2	True	False	Add risk premium only
P3	False	True	Add coverage premium only
P4	False	False	Add neither surcharge

Although there are three independent paths in the CFG, there are four possible combinations of the two decisions.

These four combinations provide complete decision-outcome coverage for the simplified logic.

7. Reconstructed Test Cases

Assume a base premium of ₹10,000, a risk surcharge of ₹2,000, and a comprehensive-coverage surcharge of ₹3,000.

Test ID	Age	Previous Accidents	Comprehensive Coverage	Expected Premium	Path
TC1	22	1	Yes	₹15,000	P1
TC2	22	1	No	₹12,000	P2
TC3	35	0	Yes	₹13,000	P3
TC4	35	0	No	₹10,000	P4

These tests exercise all four combinations of the two decisions.

Testing the Compound Condition

For the condition:

age < 25 && accidents > 0

use the following tests:

Test ID	Age	Previous Accidents	age < 25	accidents > 0	Overall Result
C1	22	1	True	True	True
C2	22	0	True	False	False
C3	30	1	False	True	False

These three tests demonstrate the important combinations needed to evaluate the effect of both conditions.

- C1 makes both conditions True.
- C2 keeps the age condition True but changes the accident condition to False.
- C3 changes the age condition to False while the accident condition remains True.

Together, they satisfy **Modified Condition/Decision Coverage (MC/DC)** for this simple two-condition decision, assuming the tests execute the decision normally.

Boundary tests should also examine age values of 24, 25, and 26, and accident counts of 0 and 1, to identify incorrect comparison operators.

8. Branch Coverage and Path Coverage

8.1 Branch Coverage

Branch Coverage requires every decision outcome to execute at least once.

For this example, the tests must exercise:

- D1 = True
- D1 = False
- D2 = True
- D2 = False

The four integrated test cases TC1–TC4 satisfy these requirements.

8.2 Path Coverage

Path Coverage checks the possible execution paths through the control flow graph.

For the simplified calculator, the four combinations are:

1. Risk surcharge added → Coverage surcharge added.
2. Risk surcharge added → Coverage surcharge not added.
3. Risk surcharge not added → Coverage surcharge added.
4. Risk surcharge not added → Coverage surcharge not added.

Testing all four combinations provides complete path coverage for this finite, loop-free example.

In more complex programs containing loops, the number of possible paths may be very large or infinite. In such cases, testers generally select a suitable set of paths rather than claiming exhaustive path coverage without evidence.

9. Improved White-Box Testing Strategy

The reconstructed strategy should include the following stages.

Stage 1: Statement Coverage

Confirm that all executable statements have been executed.

Stage 2: Branch Coverage

Test both True and False outcomes of every decision.

Stage 3: Condition Coverage

Make every individual Boolean condition True and False at least once.

Stage 4: MC/DC Testing

For safety-critical or high-assurance premium calculations, demonstrate that each individual Boolean condition can independently affect the overall decision outcome.

Stage 5: Path Testing

Execute all four decision combinations in the simplified calculator.

Stage 6: Boundary Value Analysis

Test age values immediately below, at, and above the age threshold, as well as accident counts at 0 and 1.

Stage 7: Calculation Verification

Compare actual premiums against independently calculated expected values. Check that surcharges are added exactly once and that repeated calculations do not retain unwanted values from previous requests.

Stage 8: Regression Testing

After correcting defects, rerun the full test suite to verify that changes do not introduce new calculation errors.

10. How the Reconstruction Reduces Calculation Errors

The improved testing strategy reduces risk in several ways:

- **Detects Boolean logic defects:** Tests expose incorrect operators, missing conditions, and improperly combined expressions.
- **Verifies premium calculations:** Expected results confirm that the correct surcharges are applied.
- **Detects boundary errors:** Tests near age thresholds help identify off-by-one mistakes.
- **Improves decision coverage:** Both True and False outcomes are exercised.
- **Reduces missed scenarios:** Independent paths and condition-level tests cover combinations overlooked by statement-only testing.
- **Supports regression testing:** Previously detected defects can be checked after code changes.

For example, if the intended condition is `age < 25 && accidents > 0` but the implementation mistakenly uses `age <= 25 && accidents > 0`, a test at age 25 can expose the defect even if statement coverage is already 100%.

11. Conclusion

The original test suite relied primarily on Statement Coverage, which was inadequate for detecting defects in multi-condition decisions. Reconstructing the CFG reveals the decision points and independent execution paths in the insurance premium calculator. Branch Coverage, Condition Coverage, MC/DC, Boundary Value Analysis, and Path Testing provide stronger evidence that the logic behaves correctly across different customer profiles. Applying these techniques together helps prevent incorrect surcharges, missed premium adjustments, and calculation errors.

9.Reverse Engineering Black-Box Testing for a Cloud-Based File Synchronization Tool

1. Introduction

A cloud-based file synchronization tool transfers files between a user's device and cloud storage while maintaining consistent copies across locations. However, defects involving file size, file type, and network speed may cause incomplete uploads, failed downloads, synchronization conflicts, or lost files. By analyzing user activity logs and defect reports, testers can infer the synchronization algorithm's validation rules and likely Equivalence Class Partitioning (ECP). A revised strategy combining ECP, Boundary Value Analysis (BVA), and stress testing can identify hidden defects and improve data integrity, reliability, and recovery.

2. Analysis of Defect Patterns

User activity logs and lost-file reports provide clues about possible synchronization failures.

Observed Defect Pattern	Possible Cause	Recommended Testing Approach
Large files disappear during upload	File-size limits, memory exhaustion, or incomplete chunk handling	BVA and large-file stress testing
Small files synchronize successfully, but empty files fail	Zero-byte files may be handled incorrectly	ECP and BVA
Certain file extensions fail to synchronize	Unsupported file types or incorrect filtering	ECP for file types
Files with special characters in their names disappear	Filename parsing or path-handling defects	Input validation and negative testing
Uploads fail on slow networks	Timeout, retry, or connection-handling problems	Network stress and interruption testing
Files become corrupted after interrupted uploads	Incomplete transfer or incorrect recovery logic	Fault-injection and integrity testing
Older versions overwrite newer versions	Incorrect conflict resolution	Concurrent-update testing
Duplicate files appear after reconnection	Retry requests are processed more than once	Idempotency and recovery testing
Synchronization stops after many files	Resource exhaustion or queue-processing failure	High-volume stress testing

These patterns are hypotheses. They should be confirmed by correlating defect reports with file metadata, network conditions, synchronization events, and observed results.

3. Decomposition of the Synchronization Algorithm

The synchronization process can be divided into the following functional components.

Component	Responsibility	Possible Defects
1. File Scanner	Identifies files that require synchronization	Missing files or incorrect file detection
2. Metadata Validator	Checks filename, file type, size, and timestamps	Incorrect rejection or acceptance
3. Change Detector	Determines whether a file has changed	Unnecessary uploads or missed changes
4. Upload Manager	Transfers local files to cloud storage	Incomplete or failed uploads
5. Download Manager	Transfers cloud files to local storage	Incomplete downloads
6. Network Handler	Manages connectivity, timeouts, and retries	Lost transfers or infinite retries
7. Conflict Resolver	Handles simultaneous file modifications	Overwritten or inconsistent versions
8. Integrity Checker	Verifies that transferred files match the originals	Undetected corruption
9. Recovery Manager	Resumes interrupted transfers	Duplicate files or unrecoverable transfers
10. Synchronization Logger	Records transfer results and errors	Missing or misleading diagnostic information

This decomposition helps locate which component is most likely responsible for each observed defect.

4. Inferring the Original Equivalence Class Partitioning

Equivalence Class Partitioning divides inputs into groups expected to behave similarly. The original developers may have grouped files according to size, type, and network condition.

4.1 File-Size Equivalence Classes

Assume, for illustration, that the tool supports files up to 100 MB.

Equivalence Class	Example	Expected Behavior
Empty file	0 bytes	Synchronize correctly if permitted
Small file	1 KB	Successful synchronization
Normal file	10 MB	Successful synchronization
Maximum-size file	100 MB	Accepted if the limit is inclusive
Oversized file	101 MB	Rejected or handled according to documented policy
Invalid size metadata	Missing or inconsistent size	Validation error or safe metadata recovery

The 100 MB limit is illustrative; the actual product limit must be established from requirements or controlled observations.

4.2 File-Type Equivalence Classes

Equivalence Class	Example	Expected Behavior
Supported document	PDF, TXT	Synchronize successfully
Supported image	PNG, JPEG	Synchronize successfully
Supported archive	ZIP	Synchronize if permitted
Unsupported file type	An extension excluded by policy	Reject or report clearly
Extensionless file	README	Handle according to policy
Misleading extension	Executable renamed as a document	Validate according to actual file-handling and security requirements
Unicode filename	Filename containing non-English characters	Preserve filename and file contents correctly
Special-character filename	Filename containing spaces or punctuation	Handle safely without path corruption

A file extension alone does not establish the true file format. If the application restricts file types for security reasons, the tests should verify the documented validation method.

4.3 Network-Speed Equivalence Classes

Equivalence Class	Network Condition	Expected Behavior
Fast network	High bandwidth and low latency	Synchronization completes normally
Moderate network	Typical bandwidth and latency	Synchronization completes within the expected time
Slow network	Low bandwidth or high latency	Transfer completes or reports a controlled failure
Intermittent network	Repeated disconnections	Transfer resumes without corruption
No network	Connection unavailable	Changes are queued or clearly reported
Unstable network	Frequent packet loss or connection resets	Retry and recovery mechanisms preserve file integrity

Justification: These classes represent distinct operating conditions without requiring every possible file size, extension, and network speed to be tested.

5. Boundary Value Analysis (BVA)

BVA focuses on values near the minimum and maximum supported limits.

Assume the maximum file size is 100 MB and the minimum permitted size is zero bytes. The actual limit and measurement convention must be confirmed.

5.1 File-Size Boundary Tests

Test ID	File Size	Purpose	Expected Result
BVA01	0 bytes	Test empty-file handling	Synchronize correctly if permitted
BVA02	1 byte	Test minimum nonempty file	Synchronize successfully

Test ID	File Size	Purpose	Expected Result
BVA03	2 bytes	Test just above minimum	Synchronize successfully
BVA04	99 MB	Test below maximum	Synchronize successfully
BVA05	100 MB	Test exact maximum	Accept if the limit is inclusive
BVA06	101 MB	Test above maximum	Reject or handle according to policy

Additional tests should be performed immediately below and above the actual configured limit. For precise testing, use the exact byte count defined by the implementation rather than rounded MB values.

5.2 Network Timeout Boundary Tests

Assume the documented network timeout is 30 seconds.

Test ID	Response Time	Expected Behavior
BVA07	29 seconds	Continue or complete normally
BVA08	30 seconds	Follow the documented timeout rule
BVA09	31 seconds	Apply timeout and recovery behavior

The test should verify whether the timeout is inclusive or exclusive and confirm that timing out does not incorrectly delete the local or cloud copy.

5.3 Retry Boundary Tests

Assume the maximum permitted consecutive retries is three.

Test ID	Retry Count	Expected Behavior
BVA10	2 retries	Retry remains permitted
BVA11	3 retries	Apply the configured limit
BVA12	4 retries	Stop, back off, or report failure according to policy

These tests help detect off-by-one errors in retry logic and prevent endless retry loops.

6. Revised Testing Strategy Integrating ECP, BVA, and Stress Testing

Phase 1: Log and Defect Analysis

Review synchronization logs and lost-file reports. Identify common characteristics such as file size, extension, network condition, upload duration, retry count, and conflict status.

Phase 2: Equivalence Partitioning

Divide file sizes, file types, and network conditions into valid and invalid classes. Select representative test cases from each class.

Phase 3: Boundary Value Analysis

Test values immediately below, at, and above file-size, timeout, and retry limits.

Phase 4: Stress Testing

Increase the workload gradually to examine system behavior under heavy use.

Stress Scenario	Test Workload	Expected Result
Large-file stress	Repeated uploads of large supported files	No missing or corrupted files
High-volume synchronization	Thousands of small files	All eligible files are synchronized
Mixed-file workload	Documents, images, and archives together	Correct handling of each supported type
Slow-network stress	Large transfers over limited bandwidth	Controlled progress, retry, or failure
Interruption stress	Disconnect during upload or download	Safe resume or restart
Concurrent synchronization	Multiple devices modify files	Defined conflict resolution
Retry stress	Repeated temporary failures	Bounded retries without duplicate files
Long-duration stress	Synchronization over several hours	Stable resource use and consistent file state

Workload sizes should be adapted to the application's expected usage and the capacity of the test environment.

Phase 5: Data Integrity Verification

After every test, compare the source and destination files using cryptographic hashes such as SHA-256. Verify that:

- File contents match after successful synchronization.
- Interrupted transfers do not replace valid files with partial content.
- Retried requests do not create unintended duplicates.
- Conflict resolution preserves the required versions.
- Logs correctly reflect success, failure, and recovery.

Phase 6: Regression Testing

After fixing a defect, repeat the original failing test and the relevant ECP, BVA, stress, and recovery tests to confirm that the defect has not returned.

7. Integrated Test Case Matrix

Test ID	Scenario	Technique	Expected Result
TC01	Synchronize an empty file	ECP/BVA	Correct handling according to policy
TC02	Synchronize a normal supported file	ECP	File matches the source
TC03	Synchronize a file at the maximum limit	BVA	Accepted if the limit is inclusive
TC04	Synchronize a file above the maximum limit	BVA	Rejected or handled safely
TC05	Synchronize an unsupported file type	ECP	Clear rejection or documented handling
TC06	Synchronize a Unicode filename	ECP	Filename and contents preserved

Test ID	Scenario	Technique	Expected Result
TC07	Upload over a slow network	ECP/Stress Testing	Completion or controlled failure
TC08	Disconnect during upload	Fault-Injection Testing	Safe recovery without data loss
TC09	Disconnect during download	Fault-Injection Testing	No partial file presented as complete
TC10	Exceed the timeout boundary	BVA	Correct timeout and recovery behavior
TC11	Exceed the retry boundary	BVA	Retry limit enforced
TC12	Synchronize thousands of files	Stress Testing	All eligible files accounted for
TC13	Modify the same file on two devices	Concurrency Testing	Documented conflict policy applied
TC14	Repeat the same synchronization request	Idempotency Testing	No unintended duplicate processing
TC15	Verify hashes after synchronization	Integrity Testing	Source and destination hashes match

8. How the Revised Approach Prevents Future Data Loss

The revised approach reduces data-loss risk in several ways:

1. **ECP improves input coverage:** It checks representative valid and invalid file types, sizes, and network conditions.
2. **BVA identifies boundary defects:** It detects errors at file-size, timeout, and retry limits.
3. **Stress testing exposes resource problems:** It reveals failures caused by large workloads, concurrent users, and long-running synchronization.
4. **Interruption testing validates recovery:** It ensures transfers can resume safely after network failures.
5. **Integrity checking detects corruption:** Hash comparisons reveal whether transferred files differ from their sources.
6. **Conflict testing protects file versions:** It verifies that simultaneous updates follow a defined conflict-resolution policy.
7. **Regression testing preserves improvements:** It confirms that fixes do not reintroduce previous synchronization defects.

These techniques reduce the likelihood of data loss but cannot guarantee its complete elimination. Reliable synchronization also requires atomic file replacement, durable metadata, safe retry behavior, conflict handling, and recoverable storage operations.

9. Conclusion

Reverse engineering a cloud synchronization algorithm from user activity logs and lost-file reports involves identifying relationships between failures, file characteristics, and network conditions. The original developers' likely Equivalence Class Partitioning can be inferred by grouping inputs into valid and invalid categories. A revised strategy that integrates ECP, Boundary Value Analysis, stress testing, fault injection, and file-integrity verification provides broader coverage. This approach helps uncover boundary defects, resource limitations, incomplete transfers, and synchronization conflicts, thereby improving reliability and reducing the risk of future data loss.

10.Decision Table Testing of an E-Commerce Search and Filter Module

1. Introduction

An e-commerce Search and Filter module allows users to find products based on conditions such as product name, category, price, brand, and availability. SQL queries generated by the module can reveal how these conditions are combined using logical operators such as AND, OR, and NOT. By analyzing query parameters and the resulting datasets, testers can reverse-engineer the filtering rules and identify missing test scenarios. Decision Table Testing provides a structured method for verifying combinations of conditions and ensuring that the module returns the correct products.

Note: Since actual SQL queries and datasets were not provided, the following analysis uses an illustrative e-commerce search model. The exact logic must be confirmed using the supplied queries and their results.

2. Reverse Engineering the Search Logic

Consider the following example SQL query:

```
SELECT *
FROM Products
WHERE category = 'Electronics'
  AND price <= 50000
  AND (brand = 'Samsung' OR brand = 'Apple')
  AND stock > 0;
```

This query identifies products that satisfy all the following requirements:

1. The product category must be Electronics.
2. The price must be ₹50,000 or less.
3. The brand must be Samsung or Apple.
4. The product must be in stock.

The query uses **AND** to require multiple conditions to be satisfied and **OR** to allow either of two brands.

Consider another query:

```
SELECT *
FROM Products
WHERE category = 'Electronics'
  AND price <= 50000
  AND NOT brand = 'Samsung';
```

This query excludes Samsung products while returning qualifying products from other brands.

Identifying the Operators

Operator	Meaning	Testing Requirement
AND	All connected conditions must be true	Test cases where each condition fails individually

Operator	Meaning	Testing Requirement
OR	At least one connected condition must be true	Test cases where either condition is true and both are false
NOT	Reverses a Boolean condition	Test included and excluded values
Parentheses	Control the order in which conditions are evaluated	Test combinations that distinguish alternative groupings

For example, the following expressions are not equivalent:

```
WHERE category = 'Electronics'
  AND (brand = 'Samsung' OR brand = 'Apple')
WHERE (category = 'Electronics' AND brand = 'Samsung')
  OR brand = 'Apple'
```

The second expression can return Apple products from categories other than Electronics. Testing such cases helps identify incorrect grouping of conditions.

3. Decomposition into Testable Components

The search module can be divided into the following functional components.

Component	Responsibility	Possible Defects
1. Search Input	Accepts search keywords	Empty or incorrectly interpreted keywords
2. Category Filter	Filters by product category	Incorrect category results
3. Price Filter	Applies minimum and maximum price	Boundary errors or incorrect comparisons
4. Brand Filter	Includes selected brands	Incorrect OR conditions
5. Exclusion Filter	Excludes specified brands or products	Incorrect NOT conditions
6. Availability Filter	Returns available products	Out-of-stock products included
7. Query Builder	Combines filtering conditions	Incorrect AND/OR grouping
8. Database Executor	Executes generated SQL	Query execution failures
9. Result Processor	Returns matching products	Missing, duplicated, or unexpected products

This decomposition helps testers associate incorrect results with the relevant filtering component.

4. Define the Conditions and Actions

Assume the search module applies four conditions:

- **C1:** Category is Electronics.
- **C2:** Price is ₹50,000 or less.
- **C3:** Brand is Samsung or Apple.
- **C4:** Product is in stock.

The expected action is:

A1: Display the product if all four conditions are satisfied.

The decision rule is:

A1=C1∧C2∧C3∧C4A1 = C1 \land C2 \land C3 \land C4

Here, ∧\land represents AND. The brand condition internally uses OR because either Samsung or Apple is acceptable.

For an individual product, each condition evaluates to True or False.

5. Decision Table

A complete combination of four independent Boolean conditions has:

24=16 combinations2^4 = 16 \text{ combinations}

The following Decision Table covers all 16 combinations.

Legend:

- T = True
- F = False
- Display = Product should appear in the results
- Exclude = Product should not appear in the results

Rule	C1: Electronics	C2: Price ≤ ₹50,000	C3: Samsung or Apple	C4: In Stock	Expected Action
R1	T	T	T	T	Display
R2	T	T	T	F	Exclude
R3	T	T	F	T	Exclude
R4	T	T	F	F	Exclude
R5	T	F	T	T	Exclude
R6	T	F	T	F	Exclude
R7	T	F	F	T	Exclude
R8	T	F	F	F	Exclude
R9	F	T	T	T	Exclude
R10	F	T	T	F	Exclude
R11	F	T	F	T	Exclude
R12	F	T	F	F	Exclude
R13	F	F	T	T	Exclude
R14	F	F	T	F	Exclude
R15	F	F	F	T	Exclude
R16	F	F	F	F	Exclude

Only R1 satisfies every required condition. The remaining rules should exclude the product.

Important: This table models whether an individual product qualifies for the result set. It does not mean that the complete search result will contain only one product; every qualifying product should be returned.

6. SQL-Based Test Cases

Assume the database contains the following sample records.

Product ID	Category	Price	Brand	In Stock
P101	Electronics	₹40,000	Samsung	Yes
P102	Electronics	₹45,000	Apple	Yes
P103	Electronics	₹55,000	Samsung	Yes
P104	Electronics	₹30,000	Sony	Yes
P105	Electronics	₹35,000	Apple	No
P106	Furniture	₹20,000	Samsung	Yes

For the query defined earlier, the expected results are:

Test ID	Test Scenario	Expected Result	Reason
TC01	Samsung product within the price limit and in stock	Include P101	All conditions are satisfied
TC02	Apple product within the price limit and in stock	Include P102	OR condition accepts Apple
TC03	Product exceeds the price limit	Exclude P103	Price condition is false
TC04	Product has an unsupported brand	Exclude P104	Brand condition is false
TC05	Product is out of stock	Exclude P105	Availability condition is false
TC06	Product belongs to another category	Exclude P106	Category condition is false
TC07	Product meets all conditions exactly at ₹50,000	Include if the upper limit is inclusive	Tests the price boundary
TC08	Product price is just above ₹50,000	Exclude	Tests the upper boundary
TC09	No products match the conditions	Return an empty result set	Tests no-match behavior
TC10	Brand filter is omitted	Follow documented default behavior	Tests optional filters

The exact boundary tests should use the currency precision and comparison rules specified by the application.

7. Testing Complex AND, OR, and NOT Logic

7.1 AND Testing

For AND conditions, each condition should be tested as false while the others remain true.

Example:

```
WHERE category = 'Electronics'
  AND price <= 50000
```

```
AND stock > 0
```

Test a qualifying product, then independently change its category, price, and stock status. Each change should cause the product to be excluded.

7.2 OR Testing

For the brand condition:

```
AND (brand = 'Samsung' OR brand = 'Apple')
```

Test:

- Samsung product: included if the other conditions pass.
- Apple product: included if the other conditions pass.
- Sony product: excluded if the other conditions pass.

This confirms that either accepted brand satisfies the OR condition.

7.3 NOT Testing

For the exclusion condition:

```
AND NOT (brand = 'Samsung')
```

Test:

- Apple product: included if the other conditions pass.
- Samsung product: excluded.
- A product with a NULL brand: verify the documented NULL-handling behavior.

In SQL, comparisons involving NULL can evaluate to UNKNOWN rather than TRUE or FALSE. Therefore, NULL handling must be tested explicitly when nullable fields are permitted.

8. Boundary Value Analysis for Price Filtering

Decision Table Testing should be combined with Boundary Value Analysis to detect price-comparison defects.

Assume the minimum price is ₹1,000 and the maximum price is ₹50,000.

Test ID	Minimum Price	Maximum Price	Product Price	Expected Result
BVA01	₹1,000	₹50,000	₹999.99	Exclude
BVA02	₹1,000	₹50,000	₹1,000	Include if minimum is inclusive
BVA03	₹1,000	₹50,000	₹1,000.01	Include
BVA04	₹1,000	₹50,000	₹49,999.99	Include
BVA05	₹1,000	₹50,000	₹50,000	Include if maximum is inclusive
BVA06	₹1,000	₹50,000	₹50,000.01	Exclude

These values are illustrative. Use the application's actual price precision and boundary requirements.

9. Evaluating the Completeness of the Reconstructed Model

The Decision Table covers all 16 Boolean combinations for the four defined conditions. However, this does not establish complete coverage of every behavior in the actual search module.

Coverage Area	Completeness	Additional Testing Required
Four-condition Boolean combinations	Complete for the defined model	Validate the inferred conditions against actual queries
AND logic	Covered	Test each condition failing independently
OR logic	Covered for the two-brand example	Test all accepted and rejected brands
NOT logic	Requires explicit exclusion tests	Test excluded values and NULL behavior
Price boundaries	Partially covered by the decision table	Apply BVA at minimum and maximum prices
Optional filters	Not fully represented	Test omitted, empty, and default filter values
SQL NULL values	Not fully represented	Verify database-specific NULL behavior
Multiple selected filters	Covered only by the assumed logic	Compare actual generated queries and result sets
Empty search results	Included as a scenario	Verify pagination and no-match messages
Sorting and pagination	Not covered	Test ordering, page boundaries, and duplicate results
Security	Not covered by Boolean combinations alone	Test parameterized queries and injection resistance

Completeness Assessment

The model is complete for the four independent Boolean conditions that were defined. However, the original query parameters and datasets are unavailable, so the reconstructed rules remain assumptions.

A stronger assessment should compare actual SQL predicates with expected results, test NULL and missing values, and verify that filter combinations behave consistently across different datasets.

10. Conclusion

Decision Table Testing is appropriate for reverse-engineering an e-commerce Search and Filter module because the output depends on combinations of multiple conditions. SQL query analysis helps identify the use of AND, OR, NOT, and parentheses, while the Decision Table systematically covers every Boolean combination in the reconstructed model. Combining this technique with Boundary Value Analysis and explicit NULL, optional-filter, and security testing improves confidence that the module returns the correct products. The final model should be validated against actual SQL queries and database results before claiming complete coverage of the original application.